From Python to Polynomial Certificate

A concrete barrier-theoretic transition system for SQL injection safety

---

A3 Python Analysis Engine

Step 1: The Python Code

webapp.py

```python
from flask import request
import sqlite3

def search_users(db):
    # ⚠ SOURCE: user input
    query = request.args.get("q")

    if query is not None:
        # BRANCH A: parameterized (safe)
        if use_safe_api():
            cursor = db.execute(
                "SELECT * FROM users "
                "WHERE name = ?", (query,)
                )

        # BRANCH B: string format (unsafe!)
        else:
```

Key observations

- `request.args.get("q")` is a **taint source**
- `db.execute(...)` is a **taint sink**
- Branch A uses parameterized query ⇒ **SAFE**
- Branch B concatenates user input ⇒ **UNSAFE**

Goal: Produce a *polynomial certificate* proving Branch A safe, and flagging Branch B.

Step 2: Extract the Transition System $\mathcal{T}$

State tuple from A3's symbolic VM

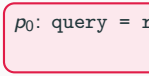
$$\sigma = (\underbrace{pc}_{\text{program counter}}, \underbrace{\rho}_{\text{locals}}, \underbrace{h}_{\text{heap}}, \underbrace{\tau}_{\text{taint map}}, \underbrace{\kappa}_{\text{call stack}}, \underbrace{e}_{\text{exception flag}}, \underbrace{\Phi}_{\text{path condition}})$$

For taint analysis, we **project** onto the relevant numeric dimensions:

$$\hat{\sigma} = (\underbrace{p}_{\substack{\text{program}\\\text{location}}}, \underbrace{\tau_q}_{\substack{\text{taint of}\\\texttt{query}}}, \underbrace{\sigma_q}_{\substack{\text{sanitized?}\\\texttt{query}}}, \underbrace{\delta}_{\substack{\text{at SQL}\\\text{sink?}}}) \in \{0, \ldots, 5\} \times \{0, 1\} \times \{0, 1\} \times \{0, 1\}$$

p: location index $\quad \tau_q \in \{0, 1\}$: taint bit $\quad \sigma_q \in \{0, 1\}$: sanitization bit $\quad \delta \in \{0, 1\}$: at SQL sink

Step 2b: The CFG as a Directed Graph



Step 2c: The Formal Transition Relation T

Transition relation (one step of symbolic execution)

$$T(\hat{\sigma}, \hat{\sigma}') := \bigvee_i \Big[\underbrace{p = p_i \ \wedge \ p' = p_j}_{\text{CFG edge}} \ \wedge \ \underbrace{G_{ij}(\hat{\sigma})}_{\text{guard}} \ \wedge \ \underbrace{U_{ij}(\hat{\sigma}, \hat{\sigma}')}_{\text{update}} \Big]$$

Edge	**Guard** G	**Update** U	**Semantics**
$p_0 \to p_1$	true	$\tau'_q = 1,\ \sigma'_q = 0,\ \delta' = 0$	Taint from source
$p_1 \to p_2$	`query` $\neq$ `None`	$\tau'_q = \tau_q$ (propagate)	Branch into handler
$p_1 \to p_5$	`query` $=$ `None`	$\delta' = 0$	Skip (no sink reached)
$p_2 \to p_3$	`use_safe_api()`	$\sigma'_q = 1,\ \delta' = 1$	Parameterized
$p_2 \to p_4$	$\neg$`use_safe_api()`	$\sigma'_q = 0,\ \delta' = 1$	Concatenation

Key: taint τ_q is set at source, propagated through operations, and checked at sinks where $\delta = 1$.

Step 3: Define Initial, Safe, and Unsafe Regions

Regions in state space

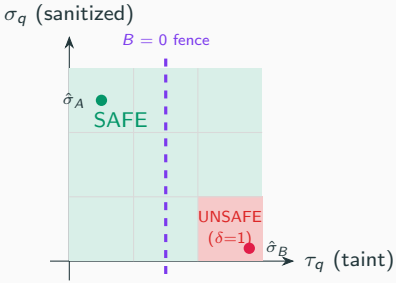
$$S_0 := \{\hat{\sigma} : p = p_0\}$$

$$U := \{\hat{\sigma} : \delta = 1 \ \wedge \ \tau_q = 1 \ \wedge \ \sigma_q = 0\}$$

$$Safe := \{\hat{\sigma} : \delta = 0 \ \vee \ \tau_q = 0 \ \vee \ \sigma_q = 1\}$$

In words:

- **Initial:** program start
- **Unsafe:** at a SQL sink, with tainted input, without parameterization
- **Safe:** complement of unsafe



$\hat{\sigma}_A$: Branch A (parameterized)
$\hat{\sigma}_B$: Branch B (string concat)

Step 3b: Encode as Polynomial Constraints

Semialgebraic encoding of regions

Since $\tau_q, \sigma_q, \delta \in \{0, 1\}$, they satisfy $x^2 = x$ (idempotent). The regions become polynomial constraints over $(\tau_q, \sigma_q, \delta)$:

$$\text{Bool constraint:} \quad \tau_q^2 - \tau_q = 0, \quad \sigma_q^2 - \sigma_q = 0, \quad \delta^2 - \delta = 0 \tag{1}$$

$$\text{Unsafe set:} \quad g_U(\hat{\sigma}) \;=\; \delta \cdot \tau_q \cdot (1 - \sigma_q) \;\geq\; 1 \tag{2}$$

$$\text{(equivalently: } \delta = 1 \wedge \tau_q = 1 \wedge \sigma_q = 0\text{)}$$

$$\text{Safe set:} \quad g_S(\hat{\sigma}) \;=\; 1 - \delta \cdot \tau_q \cdot (1 - \sigma_q) \;\geq\; 0 \tag{3}$$

Boolean program variables become degree-1 polynomials.
Set membership becomes polynomial ≥ 0 constraints.
This is exactly the **Positivstellensatz** encoding.

Step 4: Construct the Barrier Certificate B

Barrier template (degree 2 in taint variables)

$$B(\tau_q, \sigma_q, \delta) \;=\; c_0 + c_1\,\tau_q + c_2\,\sigma_q + c_3\,\delta + c_4\,\tau_q\sigma_q + c_5\,\tau_q\delta + c_6\,\sigma_q\delta$$

Three obligations (from A3's barrier API, per Prajna–Jadbabaie 2004):

$$\text{(Init)} \quad \forall \hat{\sigma} \in S_0 : \; B(\hat{\sigma}) \geq \epsilon \tag{4}$$

$$\text{(Consecution)} \quad \forall \hat{\sigma}, \hat{\sigma}' : \; B(\hat{\sigma}) \geq 0 \;\wedge\; T(\hat{\sigma}, \hat{\sigma}') \;\Rightarrow\; B(\hat{\sigma}') \geq 0 \tag{5}$$

$$\text{(Separation)} \quad \forall \hat{\sigma} \in U : \; B(\hat{\sigma}) \leq -\epsilon \tag{6}$$

Solve for $c_0, \ldots, c_6$ using Z3 or SOS relaxation.

Step 4b: Concrete Solution

Synthesized certificate (from A3 barrier engine)

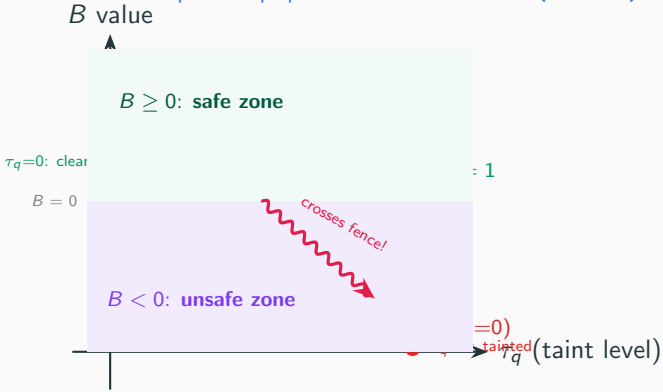
$$\boxed{B(\tau_q, \sigma_q, \delta) \;=\; 1 \;-\; 2\,\delta\,\tau_q \;+\; 2\,\delta\,\tau_q\,\sigma_q}$$

Verify the three obligations:

Obligation	Check	Result
Init ($p = p_0$, so $\delta = 0$)	$B = 1 - 0 + 0 = 1 \geq \epsilon$	✔
Separation ($\delta{=}1, \tau_q{=}1, \sigma_q{=}0$)	$B = 1 - 2 + 0 = -1 \leq -\epsilon$	✔
Consecution (Branch A: $\sigma'_q{=}1$)	$B' = 1 - 2 + 2 = 1 \geq 0$	✔
Consecution (Branch B: $\sigma'_q{=}0$)	$B' = 1 - 2 + 0 = -1 < 0$	⚠

The Polynomial Fence, Visually

$B = 1 - 2\delta\tau_q + 2\delta\tau_q\sigma_q$ **evaluated at** $\delta = 1$ **(at sink)**



Why a Barrier Certificate? (Not Type Systems, Not Model Checking)

Type systems / taint types

- Require annotations on every function boundary
- Cannot express "taint is neutralized via parameterized query at same call site"
- No notion of *consecution*: cannot reason about taint evolution across a loop
- Binary: safe/unsafe, no certificate of *degree* of separation

Explicit-state model checking

- Must enumerate all reachable states
- Taint $\times$ sanitization $\times$ call stack $=$ exponential state space
- No compact proof artifact: "I checked all states" is not compositional
- Cannot handle **infinite** domains (numeric constraints on taint propagation)

The Fundamental Difference: Information Flow is a Hyperproperty

Why standard invariants fail

SQL injection safety is an **information-flow property**: the assertion is not about a single state, but about the *relationship* between the source of a value and how it is used at a sink.

Standard inductive invariant

$$Inv(s) \Rightarrow Inv(s')$$

generalize

Barrier certificate

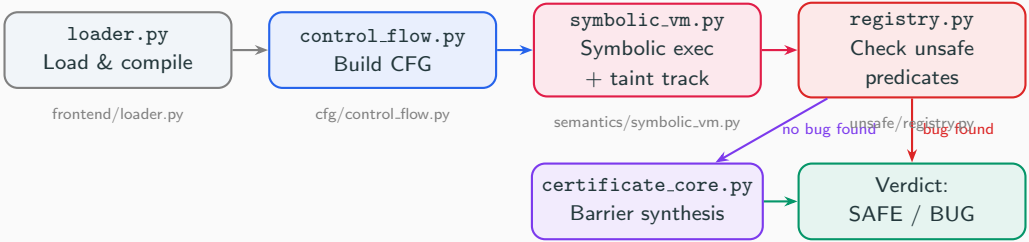
$$B(s) \geq 0 \;\wedge\; T(s, s') \;\Rightarrow\; B(s') \geq 0$$

Invariant is a *set*: $\{s : Inv(s)\}$.
Must be discovered. No template search.

Certificate is a *function*: template $\sum c_\alpha x^\alpha$.
Coefficients found by SOS/SMT.
Tracks taint quantitatively.

The barrier encodes taint, sanitization, and sink-reachability as *continuous dimensions*, allowing polynomial separation where set-based reasoning requires exponential case

How A3 Implements This End-to-End



1. `loader.py` compiles Python → bytecode
2. `control_flow.py` extracts the CFG with edge types (normal, exception, conditional)
3. `symbolic_vm.py` runs symbolic execution; `security_tracker.py` propagates taint
4. `registry.py` checks 108 unsafe predicates (SQL injection: CWE-089)
5. If no violation found on safe path: `certificate_core.py` synthesizes a barrier certificate to **prove** safety via SOS

Step-by-Step: From Bytecode to Certificate

1. Compile: `search_users` $\rightarrow$ 23 bytecode instructions

2. CFG: 6 basic blocks, 5 edges (2 conditional, 3 fallthrough)

3. Taint: $\tau(\texttt{query}) = 1$ at `request.args.get`; propagate through concat

4. Project: $\hat{\sigma} = (p, \tau_q, \sigma_q, \delta) \in \{0..5\} \times \{0,1\}^3$

5. Synthesize: $B = 1 - 2\delta\tau_q + 2\delta\tau_q\sigma_q$ via Z3 template solving

6. Verify: Init ✔ Consecution(A) ✔ Separation ✔ Consecution(B) 🐞

The Mathematics: Positivstellensatz Encoding

Putinar's Positivstellensatz (the engine under the hood)
If $B(\mathbf{x}) > 0$ on the semialgebraic set $\{g_1 \geq 0, \ldots, g_m \geq 0\}$, then:

$$B = s_0 + s_1 g_1 + \cdots + s_m g_m \quad \text{where each } s_i \text{ is a } \textbf{sum-of-squares (SOS)} \text{ polynomial}$$

For our SQL injection barrier, the obligations become:

$$\text{(Init on } S_0\text{):} \quad B\big|_{p=p_0} = s_0^{(\mathrm{init})} + s_1^{(\mathrm{init})} \cdot (5 - p) \quad \text{with each } s_i \text{ SOS} \tag{7}$$

$$\text{(Sep on } U\text{):} \quad -B\big|_U = s_0^{(\mathrm{sep})} + s_1^{(\mathrm{sep})} \cdot g_U \quad \text{with each } s_i \text{ SOS} \tag{8}$$

$$\text{(Consec):} \quad B(\hat{\sigma}') = s_0^{(e)} + s_1^{(e)} \cdot B(\hat{\sigma}) + s_2^{(e)} \cdot G_e(\hat{\sigma}) \quad \text{per edge } e \tag{9}$$

Search for SOS multipliers reduces to **semidefinite programming (SDP)**, which A3

Verification via Z3: The Concrete Queries

What A3 actually sends to Z3

For each obligation, A3 negates and checks unsatisfiability:

Init check (should be UNSAT):

$$\begin{aligned}\exists\, \tau_q, \sigma_q \in \{0,1\} : \quad & \tau_q^2 = \tau_q \;\wedge\; \sigma_q^2 = \sigma_q \\ & \wedge\; \delta = 0 \quad \text{(at entry, not at sink)} \\ & \wedge\; B(\tau_q, \sigma_q, 0) < \epsilon\end{aligned}$$

Separation check (should be UNSAT):

$$\begin{aligned}\exists\, \tau_q, \sigma_q : \quad & \delta = 1 \;\wedge\; \tau_q = 1 \;\wedge\; \sigma_q = 0 \\ & \wedge\; B(1,0,1) > -\epsilon\end{aligned}$$

Consecution check per edge (should be UNSAT):

$$\exists\, \hat{\sigma}, \hat{\sigma}' : \quad B(\hat{\sigma}) \geq 0 \;\wedge\; T_e(\hat{\sigma}, \hat{\sigma}') \;\wedge\; B(\hat{\sigma}') < 0$$

$\Rightarrow$ If all three return UNSAT, the certificate is **valid**.

Comparison: Three Approaches to SQL Injection Detection

Taint Type System

Annotate:

`q: Tainted[str]`

✔ Fast, local
✘ Manual annotations
✘ No loop reasoning
✘ No proof artifact

Model Checking (explicit-state)

Enumerate all paths

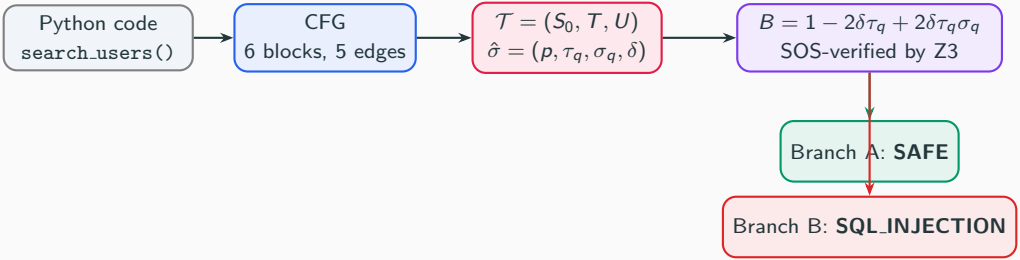
✔ Complete for finite
✘ State explosion
✘ No compact proof
✘ Not compositional

Barrier Certificate (A3 approach)

Polynomial B separates safe from unsafe

✔ Compact proof
✔ Handles loops
✔ Quantitative
✔ Compositional

Summary: The Complete Pipeline



Key takeaway

A **polynomial barrier certificate** gives a compact, verifiable, compositional proof that tainted data cannot reach SQL sinks through safe paths — something neither type systems nor explicit-state model checking can provide for programs with loops, dynamic dispatch, and infinite input domains.

fin

Python $\rightarrow$ CFG $\rightarrow$ Transition System $\rightarrow$ Polynomial $\rightarrow$ Proof

Built on: Prajna–Jadbabaie (barrier certificates), Putinar (Positivstellensatz), Z3 (SMT)